Camerala: A Local-First Browser-Native Framework for Remote Psychophysiological Task Sensing

Kotaro Furukawa*1

Abstract – **Camerala is a local-first, browser-native framework for remote psychological and psychophysiological task experiments. It synchronizes task events with face-image-derived feature records and exports local CSV/ZIP files without transmitting or storing raw facial video. In an exploratory N=1 author-participant deployment, ten sessions were conducted in laboratory and home settings, yielding 600 trials in total. After applying a reaction-time criterion of 100–1500 ms, 541/600 trials remained for descriptive analysis. The exported logs contained timestamped task events and reproducible face-image-derived fields. Differences between laboratory and home exported records are reported as deployment-context differences in face-derived and sensor-derived records, not as evidence of psychological, physiological, cognitive, motivational, or rPPG-accuracy effects.**

Keywords : **Browser-Native Sensing, Remote Photoplethysmography, FaceMesh, Local-First Logging, Task-Aligned Measurement, Laboratory and Home Settings**

1. Introduction

Remote psychological and psychophysiological task experiments often require synchronized behavioral task logs and webcam-derived feature records while avoiding unnecessary handling of privacy-sensitive facial video. This use case is not unconstrained daily-life sensing: the participant is seated in front of a laptop, views task stimuli, and responds through the browser. Even in this task-constrained setting, deployment outside a controlled laboratory introduces practical difficulties for human-interface research, including heterogeneous illumination, camera placement, posture, browser timing, and local data handling.

Many video-based physiological measurement pipelines rely on raw video recording or server-side processing. Such approaches can be difficult to deploy in private spaces because raw facial video is privacy-sensitive and may be identifiable. A local-first browser architecture offers an architectural privacy advantage: raw frames can be processed on the client device, while only extracted features, task events, and quality indicators are stored. This does not constitute a dedicated privacy-preserving algorithm; rather, it is a system-design choice that reduces the need to transmit or retain raw video.

Real-time processing is not treated as the only technically possible architecture. Local video recording followed by offline processing could preserve timestamps if implemented carefully. However, the intended use case of Camerala is a remote psychological or psychophysiological task experiment conducted in a private space, where retaining identifiable facial video for later analysis can itself be a privacy and acceptability burden. Camerala therefore extracts face-image-derived records during task execution and stores only task events, exported feature fields, and auxiliary context fields, without raw facial video persistence. In this sense, real-time browser-side processing is used to support feature-only local logging and timestamp alignment within the same client-side runtime, rather than to claim more accurate physiological estimates or online task adaptation.

To address this deployment scenario, we present **Camerala**, a browser-native framework for task-aligned physiological feature logging in remote task experiments conducted in laboratory and home settings.

1.1 Research Questions

The exploratory Research Questions (RQs) focus on task-synchronized face-derived records and descriptive sensor-derived observations under the tested configuration:

- **RQ1:** In the intended remote psychological/psychophysiological task use case, do Camerala's task-synchronized face-derived records,

*1: College of Transdisciplinary Sciences for Innovation, Kanazawa University, Japan

including rPPG-oriented ROI values, satisfy the operational criteria for descriptive analysis: trial-level reaction times within 100–1500 ms and exported-window-level landmark availability of `mean_quality` $\geqq 0.8$ under the tested configuration?

- **RQ2:** What kinds of between-environment differences are observable in the physiological signals collected through such a deployment?

For RQ1, we evaluated the exported records using operational criteria at the trial and exported-window levels. At the trial level, trials were retained for descriptive analysis when reaction times fell within 100–1500 ms. At the exported-window level, landmark availability was evaluated using `mean_quality`, the mean of the binary landmark-availability flag within each 10-s window; exported windows with `mean_quality` $\geqq 0.8$ were treated as satisfying the operational landmark-availability criterion. This criterion describes the exported frame/window records and should not be interpreted as face-detection availability across all camera frames. We also examined whether the exported records contained reproducible face-derived fields, including rPPG-oriented ROI values, motion estimates, and a frame-to-frame green-channel brightness-change indicator. In RQ2, the term "physiological signals" is used operationally to refer to task-synchronized face-derived and sensor-derived exported records, including rPPG-oriented ROI values and related exported fields. These records are not treated as validated physiological-state measurements. The present design was not able to determine whether laboratory/home differences reflected psychological state, physiological state, lighting, screen light, posture, camera geometry, the measurement pipeline, task timing, participant state, or their mixture. Therefore, laboratory/home differences were interpreted as deployment-context differences rather than psychological, cognitive, motivational, or physiological effects. These analyses are intended only to describe operational completion and exported-record differences under the tested configuration; they do not demonstrate general deployability across users, devices, or environments.

1.2 Contributions

In this paper, we focus on system integration and conservative deployment observations. The contributions are:

1. **Local-first browser-native system-integration implementation**: An implementation for remote psychological and psychophysiological task experiments that combines task execution, face-derived feature extraction, timestamp alignment, and local persistence within one browser runtime.
2. **Task-synchronized feature logging pipeline**: A logging pipeline that records task events, rPPG-oriented ROI values, FaceMesh-derived motion estimates, operational landmark-availability ratios, and frame-derived brightness-change indicators without raw facial video persistence.
3. **Exploratory deployment under limited conditions**: An N=1 author-participant deployment showing that, under the tested configuration, the exported records satisfied the specified reaction-time and landmark-availability criteria for descriptive analysis.

The remainder of this paper is organized as follows. Section II reviews related work in rPPG, web/edge physiological sensing, and task-measurement platforms. Section III describes the Camerala system architecture. Section IV describes the task-constrained deployment and reporting details. Section V presents descriptive observations. Section VI discusses design requirements and safeguards for future deployments, and Section VII concludes.

2. Related Work

2.1 Remote Physiological Measurement and rPPG

Remote photoplethysmography (rPPG) estimates pulse-related color changes from facial video. Classic approaches include ambient-light remote plethysmographic imaging and blind-source video methods [1], [2], while more recent work includes deep-learning-based estimators [3]. These studies show that camera-based and webcam-based rPPG functionality already exists. They also show that uncontrolled capture conditions, implementation choices, and evaluation pipelines are important factors in remote heart-rate imaging. Accordingly, Camerala does not claim to introduce a new rPPG estimator or to solve rPPG degradation under uncontrolled conditions. It records rPPG-oriented ROI values and quality indicators so that task-aligned observations

can be inspected and filtered.

2.2 Browser-Based, Web-Deployable, Edge-Based, and Privacy-Oriented rPPG Systems

Several closely related prior studies and systems are directly relevant to Camerala's positioning. Ayesha *et al.* proposed a web application for experimenting with and validating remote measurement of vital signs [4]. LabVanced provides remote heart-rate detection/rPPG functionality within an online experiment platform [5]. FacePhys-Demo demonstrates browser-based rPPG monitoring with local inference in a public web demo/repository [6]. RhythmEdge addresses contactless heart-rate estimation on edge devices [7]. Gupta and Etemad investigated privacy-preserving remote heart-rate estimation from facial videos [8]. Di Lernia *et al.* evaluated rPPG via online webcams under uncontrolled conditions [9]. Additional peer-reviewed work on open-source implementations, efficient/mobile sensing, rPPG benchmarking, and privacy-related signal removal provides supporting context [10]~[14]. These related studies and systems indicate that browser-based, web-deployable, edge-based, and privacy-oriented rPPG have already been explored. Camerala therefore does not claim novelty in browser-based rPPG functionality, local rPPG processing, edge-oriented heart-rate estimation, or privacy preservation as an algorithm. Instead, Camerala is positioned as a system-integration implementation for remote psychological and psychophysiological task experiments: it combines browser-based task execution, face-derived feature extraction, task-event/frame-record timestamp alignment, operational landmark-availability checking, and local CSV/ZIP persistence without raw facial video retention within one browser runtime.

2.3 Task Measurement and Physiological Data Collection Platforms

Well-established web experiment libraries and experiment runtimes, such as jsPsych [15] and PsychoPy/PsychoJS/Pavlovia [16], support browser-based task execution and online experiments. However, they do not generally provide an integrated webcam-derived physiological feature pipeline, task-sensing synchronization, and local-first feature logging as a single browser runtime. Conversely, cloud analytics APIs can provide rich video-based analysis but often require transmitting video streams away from the participant's device. Table 1 maps this design trade-off space.

We structure the initial evaluation as a single-case exploratory deployment. The deployment is not intended to estimate population-level psychological or physiological effects. Instead, it documents whether the integrated browser runtime can complete the planned task-constrained logging procedure and what measurement-quality differences are observed in the tested laboratory and home contexts.

3. System Architecture: Camerala

Camerala (derived from "Camera" + "Laboratory") is positioned as a local-first, browser-native research implementation for remote psychological and psychophysiological task experiments. Its contribution is not browser-based rPPG, local rPPG processing, or privacy preservation as an algorithm. Rather, it integrates task execution, face-derived feature extraction, task-event/frame-record timestamp alignment, operational landmark-availability checking, and local CSV/ZIP persistence without raw facial video retention within one browser runtime. Although the deployment in this paper used one Gabor-orientation task, the runtime separates task logic from the sensing pipeline so that other task procedures can be implemented without changing the feature-extraction loop.

3.1 High-Level Architecture

The system enables a task-constrained experimental loop inside the browser sandbox (Fig. 1). The architecture consists of three coupled modules:

1. **Task Engine**: Stimulus rendering, trial logic, key responses, feedback, staircase updates, and reaction-time logging.
2. **Vision Pipeline**: Webcam ingestion, MediaPipe Face Mesh inference, rPPG-oriented ROI feature extraction, head-motion estimates, blink/EAR features, and frame-level quality indicators.
3. **Data Manager**: Timestamp alignment, trial/frame aggregation, CSV serialization, ZIP packaging, and local persistence without raw-video storage.

Table 1 Design Trade-off Mapping of Data Collection Platforms

System / Platform	On-device video processing	Task-sensing synchronization	Local-first persistence	Raw video export	Typical use
Web experiment libraries	N/A (task only)	Partial	Depends on setup	N/A	Behavioral tasks
Experiment runtimes	Depends on deployment	Yes for task events	Depends on setup	Depends	Lab/online experiments
Browser or web rPPG systems	Often possible	Usually not task-centered	Varies	Varies	Vital-sign estimation
Cloud sensing APIs	No	Low or asynchronous	No	Often required	Video analytics
Camerala	Client-side feature extraction	Task events and frame features in one runtime	Local ZIP export of logs	Not used by Camerala	Remote task studies

Note: The table summarizes design emphasis rather than categorical superiority. Capabilities depend on implementation and deployment configuration.

3.2 Technical Implementation: The Vision Pipeline

The current implementation uses MediaPipe Face Mesh as a software framework for browser-based facial landmark extraction [17]. The camera stream is requested through `getUserMedia` with ideal constraints of 640×480 pixels and 30 fps. Because actual camera settings can depend on the browser and device, these constraints should be interpreted as requested settings rather than verified hardware settings.

3.2.1 Feature Extraction Logic

The present implementation records sensor-derived features and quality indicators rather than validated clinical or medical measurements. Two categories are central to this paper:

1. Head Motion Index (M_t) Head motion is approximated from the frame-to-frame displacement of a facial landmark. Let $L^t = (x^t, y^t)$ be the normalized landmark coordinate at frame t. The motion scalar is computed as:

$$M_t = ||L^t - L^{t-1}||_2. \tag{1}$$

This feature is used as a motion-related quality and behavior indicator. It is not an algorithmic correction for motion artifacts in rPPG.

2. ROI Green-Channel Value and Brightness-Change Indicator The implementation samples a small forehead region around a FaceMesh landmark and records the green-channel brightness as an rPPG-oriented ROI value. It also computes a frame-to-frame green-channel brightness-change indicator:

$$B_\Delta(t) = |G_t - G_{t-1}|, \tag{2}$$

where G_t is the downsampled frame-level green-channel average at frame t. Camerala does not attempt to correct rPPG degradation caused by motion or illumination changes. Instead, it records motion-related and brightness-related indicators for descriptive inspection of exported records. The brightness-change indicator does not measure camera exposure settings, external illuminance, specific environmental events, psychological or physiological state, or rPPG accuracy.

3.3 Technical Implementation: The Task Engine

The experimental psychophysical probe is a two-alternative forced-choice orientation discrimination task. Using the HTML5 Canvas API, the task engine procedurally renders oriented grating stimuli, records key responses, calculates reaction time with `performance.now()`, and updates task difficulty. Procedural rendering avoids network-dependent stimulus loading during a trial and keeps task timing in the same browser runtime as feature logging.

3.3.1 Timing and Synchronization

The implementation uses the following timing strategy:

1. **High-resolution timestamps**: Task events and frame features are timestamped using `performance.now()`.
2. **Frame-loop alignment**: Feature extraction and task rendering are coordinated with the browser frame loop.
3. **Local runtime alignment**: Task events and sensor-derived features are logged in the same browser runtime, avoiding server-side reconciliation.

3.4 Data Management and Local-First Persistence

The current implementation uses JSZip to export local log files as a ZIP archive. The archive contains

Algorithm 1 Camerala Task Loop (Simplified)

Initialize Camera, FaceMesh, Canvas
StartContinuousVisionFrameLoop()
while Session Active **do**
  $Block \leftarrow$ GetCurrentBlock()
  $Context \leftarrow Block.instruction$
  ShowInstruction(Context.text)
  for trial $i = 1$ to n **do**
    $\theta \leftarrow$ SampleOrientation({-45°, +45°})
    ShowFixation(400–600 ms)
    $t_{onset} \leftarrow$ performance.now()
    DrawOrientationStimulus(θ, 200 ms)
    while No KeyPress **do**
      Poll F/J KeyPress
    end while
    $t_{response} \leftarrow$ performance.now()
    $RT \leftarrow t_{response} - t_{onset}$
    LogTaskEvent(RT, Accuracy, Context)
    UpdateStaircase(Accuracy, relative ±5% contrast step)
    LinkTaskEventToFrameTimestamps(t_{onset}, $t_{response}$, Context)
  end for
end while

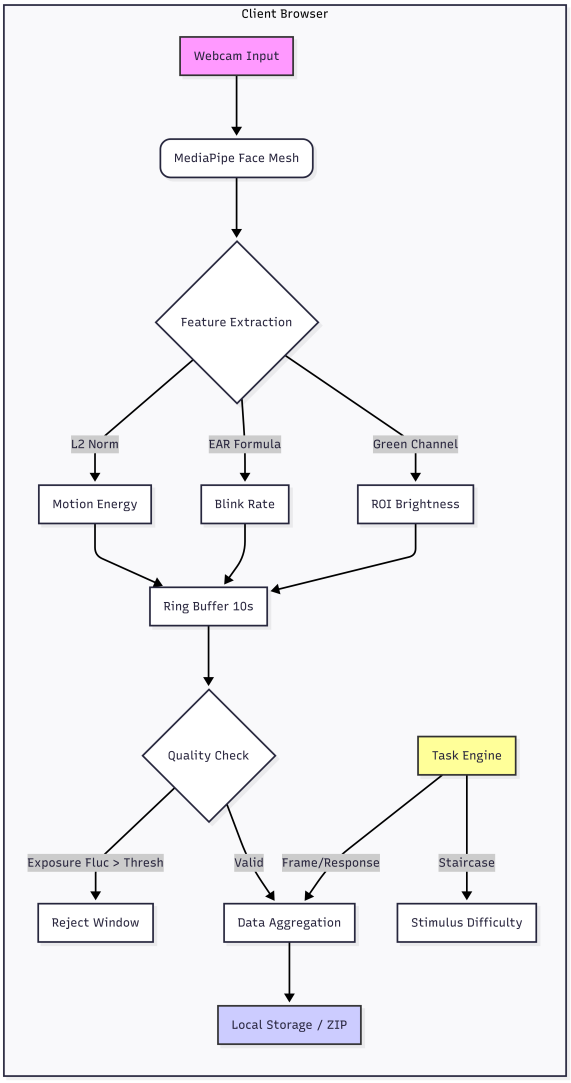

Fig. 1 Overview of the Camerala system architecture. The browser obtains webcam frames, extracts FaceMesh/rPPG-oriented features and quality indicators, synchronizes them with task events, and exports local logs. Under the tested implementation, only extracted features and task events are logged; raw video is neither transmitted nor stored by Camerala.

`metadata.json`, `trials.csv`, `features_raw.csv`, `windows.csv`, `blocks.csv`, and `subjective.csv`. The metadata file stores the session ID, start time, user agent, and block plan. The user initiates the download at the end of the session. This design avoids Camerala-side raw-video storage and avoids raw-video transmission, but it also reduces the ability to inspect image artifacts retrospectively. For this reason, Camerala logs reproducible exported fields such as rPPG-oriented ROI green-channel values,

head-motion estimates, and a frame-to-frame green-channel brightness-change indicator.

4. Methodology

4.1 Target Usage Scenario and Exploratory Deployment

The target use case is a remote psychological or psychophysiological task experiment conducted while the participant is seated in front of a laptop. It is not intended to represent continuous sensing during ordinary daily activities such as walking, cooking, conversation, or sleep. The tested deployment therefore concerns a task-constrained experiment conducted in laboratory and home settings, with browser-based logging of behavioral events and webcam-derived features.

To evaluate operational completion under this scenario, we conducted an exploratory longitudinal deployment with one author-participant. The deployment consisted of 10 sessions and 600 total trials. Each session consisted of six 10-trial blocks, with two blocks for each instructional task context (neutral instruction, negative instruction, and positive instruction), as reflected in the exported block plans. The order of blocks was randomized by the web application.

4.2 Participant (Autoethnography)

The participant was the primary author. This was a self-experimentation evaluation: no other participants were involved, no identifiable raw video was transmitted by Camerala, and the author provided informed consent for self-participation. Because the participant was also the author and knew that the task instructions were experimental, the instruction blocks cannot be assumed to have induced evaluation pressure, reward motivation, or psychological effects. They are used only as different instructional task contexts for demonstrating synchronized logging across task conditions.

4.3 Hardware and Software Configuration

Table 2 reports the hardware/software configuration used in the deployment.

4.4 Deployment Environment and Lighting Conditions

The deployment was conducted during daytime in an indoor university laboratory/research room and an indoor private home room under general artificial room lighting. Both rooms had windows, but curtains were closed during the sessions. Direct natural light was not used as the primary illumination source in either context. The built-in laptop display brightness was fixed across sessions. Because the deployment used camera-derived setup checks rather than external light-meter measurements, this paper reports brightness ranges only as author-confirmed setup/check information. The camera-derived brightness range observed during setup/checks was 105–114 in the laboratory setting and 102–111 in the home setting. Table 3 summarizes the deployment environment and lighting conditions.

4.5 Experimental Procedure and Instructional Task Contexts

Each session began with a pre-session capture check for basic camera conditions, including approximate camera-derived brightness and pose, before the camera feed was hidden to reduce distraction. Before each block, the system presented one of three instructional task contexts: neutral instruction, negative instruction, or positive instruction. The negative and positive instruction blocks were not interpreted as validated cognitive or motivational manipulations. They were used to demonstrate that Camerala can synchronize behavioral logs and sensor-derived features across different task contexts.

4.6 Psychophysical Task

The task was a two-alternative forced-choice orientation discrimination task using procedurally rendered Gabor stimuli. Each trial began with a randomized fixation period of 400–600 ms, followed by a Gabor stimulus oriented at either $+45°$ or $-45°$ for 200 ms. The participant responded using the F/J keys: the F key corresponded to $-45°$, and the J key corresponded to $+45°$. The application recorded stimulus onset, response key, reaction time, correctness, and task difficulty together with frame-level sensor-derived features. To maintain adaptive task difficulty, the implementation used a standard 1-up 1-down staircase procedure on stimulus contrast. Stimulus contrast was adjusted by relative $\pm 5\%$ steps according to the previous response. These task parameters are reported for reproducibility; the instruction blocks are not interpreted as validated cognitive or motivational manipulations.

Table 2 Hardware and software configuration

Item	Reported value
Laptop/PC model	mouse Computer G-Tune P517G60ZO21CNHWT, 15.6-inch gaming notebook
CPU	Intel Core i7-13620H
GPU	NVIDIA GeForce RTX 5060 Laptop GPU
RAM	32 GB
Storage	1 TB SSD
Operating system	Windows 11 Home 64-bit
Browsers used	Mozilla Firefox and Google Chrome
Recorded browser metadata	Session metadata stored browser user-agent strings. Mozilla Firefox and Google Chrome were used in the deployment, but browser effects were not independently manipulated or analyzed.
Webcam type/model	Built-in laptop webcam
Requested camera constraints	`getUserMedia`: ideal 640×480 pixels, ideal 30 fps
Camera capture behavior	Browser-negotiated capture under the requested constraints; no manual exposure or white-balance constraints were applied
Processing rate	Not analyzed as a result metric in the present revision
Display	Built-in 15.6-inch display, 2560×1440 pixels, 165 Hz
Screen brightness	Fixed across sessions using the built-in laptop display setting
Viewing distance	Approximately 50–60 cm
Auto-exposure	Device/browser default camera pipeline; no manual exposure control was applied
Auto-white-balance	Device/browser default camera pipeline; no manual white-balance control was applied

Table 3 Deployment environment and lighting conditions

Item	Laboratory	Home
Room type	Indoor university laboratory/research room	Indoor private home room
Time of day	Daytime	Daytime
Primary lighting	General indoor artificial lighting	General indoor artificial lighting
Window presence	Windows present	Windows present
Curtain/blind condition	Curtains present and closed	Curtains present and closed
Natural light contribution	Direct natural light was not used as the primary illumination source	Direct natural light was not used as the primary illumination source
Camera-derived brightness range observed during setup/checks	105–114	102–111
External light-meter measurement	External light-meter values were not used; the reported ranges are camera-derived setup/check information, not lux measurements.	
Screen brightness	Fixed across sessions using the built-in laptop display setting.	
Task display	Stimuli were presented on the built-in laptop display. Screen luminance was part of the tested configuration rather than an independently manipulated factor.	
Window aggregation	Exported window-level features used 10-s windows with 5-s overlap.	
Brightness-change indicator aggregation	The frame-to-frame green-channel brightness-change indicator was summarized in 10-s windows with 5-s overlap.	
Clouds/screen flicker/power cycles	Not treated as observed experimental events; possible brightness variation was handled only as camera-image-derived record variation.	

4.7 Dependent Measures and Preprocessing

All exported logs were processed sequentially. The primary behavioral measure was reaction time (RT), with an operational analysis range of 100–1500 ms. The primary sensor-derived measures were rPPG-oriented ROI green-channel values, head-motion estimates, and the frame-to-frame green-channel brightness-change indicator. Behavioral trials were retained when reaction times fell within 100–1500 ms. Camerala used MediaPipe FaceMesh to obtain facial landmarks in the browser. Frame records forwarded from the FaceMesh callback were synchronized with task events during active sessions. The feature extractor defines a binary landmark-availability flag, `quality`, where 1 indi-

cates that FaceMesh landmarks were returned and 0 indicates that no landmarks were returned. In the current runtime, however, the camera manager forwards results to the logging callback only when FaceMesh landmarks are returned; therefore, the saved frame records in this deployment represent landmark-returned exported frames. For each exported 10-s window, `mean_quality` was computed as the mean of the `quality` flag over the exported frame records in that window. Thus, `mean_quality` describes landmark availability within exported frame/window records and not face-detection availability across all camera frames. This value is not a continuous MediaPipe confidence score, an indicator of rPPG measurement quality, or evidence of physiological measurement accuracy. The FaceMesh `minDetectionConfidence` and `minTrackingConfidence` parameters were both set to 0.5 as implementation settings. They were not used as post-hoc analysis thresholds.

5. Results

The results are reported descriptively as sensor-derived observations under the tested configuration. The deployment completed all 10 planned sessions and 600 trials in total. After applying the reaction-time criterion of 100–1500 ms, 541/600 trials were retained for descriptive analysis. The retained trials were 256/300 in the laboratory setting and 285/300 in the home setting. All 299 exported windows had `mean_quality` = 1.0 and therefore satisfied the operational landmark-availability criterion of `mean_quality` $\geqq$ 0.8; no window was excluded by this criterion. These results describe the available analyzed trials and exported windows under the tested configuration and do not demonstrate general feasibility across devices, users, or environments.

5.1 Task and Exported Feature Summaries

Table 4 summarizes task-level and exported feature summaries in the laboratory and home contexts. After RT exclusion, mean RT was 721.1 ± 257.7 ms in the laboratory setting and 676.5 ± 249.5 ms in the home setting. Accuracy was 85.5% and 82.1%, respectively. Window-level exported records showed descriptive differences in ROI green-channel values, motion estimates, and the brightness-change indicator between settings. Because lighting, screen light, posture, camera geometry, task timing, and participant state were not independently controlled, these differences are reported only as deployment-context differences in exported records.

5.2 Task-Event and Sensor-Derived Observations

To address RQ2, we compared descriptive task-synchronized records between the laboratory and home contexts. The exported records showed differences in rPPG-oriented ROI green-channel values, motion estimates, and the frame-to-frame green-channel brightness-change indicator. Because lighting, screen light, posture, camera geometry, task timing, and participant state were not independently controlled, these differences are reported only as deployment-context differences in exported records. They should not be interpreted as evidence that the instruction blocks induced psychological, cognitive, motivational, or physiological effects.

In the laboratory context, the negative instruction block showed a lower mean RT than the neutral block in this dataset. In the home context, the mean RT values across instruction blocks were closer together. Because the deployment used one author-participant and did not isolate instruction effects from environment, lighting, screen light, posture, camera geometry, or measurement-pipeline factors, these RT patterns are reported only as descriptive task-event observations.

5.3 rPPG-Oriented ROI Estimates

The exported data include rPPG-oriented ROI brightness values and related quality indicators, but no ECG or contact PPG reference was collected. Therefore, this paper does not interpret ROI-derived or block-level summaries as validated heart-rate changes. The appropriate interpretation is that the tested deployment yielded face-derived and sensor-derived exported records whose values differed descriptively between the tested laboratory and home contexts.

6. Discussion

The main implication of this work is system-design oriented. Camerala illustrates how a browser-native runtime can combine task events, webcam-derived features, quality indicators, and local persistence without raw-video transmission in a remote task experiment conducted in laboratory and home settings.

Table 4 Descriptive task and exported feature summaries after reaction-time exclusion.

Metric	Laboratory	Home	Notes
Retained trials after RT exclusion	256/300 (85.3%)	285/300 (95.0%)	Trial-level; RT 100–1500 ms
Mean RT after RT exclusion	721.1 ± 257.7 ms	676.5 ± 249.5 ms	Trial-level retained trials
Accuracy after RT exclusion	85.5%	82.1%	Trial-level retained trials
rPPG-oriented ROI green-channel value	46.69 ± 6.80	34.32 ± 2.72	Window-level; windows.csv
Motion estimate	0.0011 ± 0.0005	0.0010 ± 0.0004	Window-level; windows.csv
Frame-to-frame green-channel brightness-change indicator	0.0504 ± 0.0509	0.0707 ± 0.0337	Window-level; windows.csv

The results do not establish general deployability, robustness, or physiological/cognitive effects. Instead, they motivate design requirements and safeguards for future local-first deployments.

6.1 Browser-Native Local-First Logging as a Design Pattern

The useful system property demonstrated here is integration. Task timing, frame-level feature extraction, quality indicators, and local export are handled in one client-side workflow. This can reduce the need for raw-video handling and server-side timestamp reconciliation. However, because raw video is not stored, researchers lose the ability to visually inspect many artifacts after data collection. This trade-off makes quality-aware logging especially important.

6.2 Quality Indicators for Laboratory and Home Deployment Contexts

The deployment suggests that future systems should log not only task performance and rPPG-oriented estimates, but also indicators that help interpret the measurement pipeline. Such indicators include landmark availability, rPPG-oriented ROI values, head-motion estimates, and camera-image-derived brightness-change records. These values can support pre-session checks and descriptive inspection. They should not be treated as proof that rPPG estimates are accurate.

6.3 Interpreting Sensor-Derived Estimates Under Environmental Heterogeneity

The laboratory/home differences observed in this deployment should be interpreted as mixed deployment-context differences. They may reflect environmental conditions, screen light, posture, camera geometry, the measurement pipeline, task timing, participant state, or their mixture. The present study was not designed to isolate psychological or physiological state changes from these factors. Future work should evaluate psychological task conditions under more controlled camera, lighting, posture, and environmental settings, ideally with reference ECG or contact PPG, multiple participants, multiple devices, and manipulation checks.

6.4 Design Safeguards for Future Camerala Deployments

Future deployments should incorporate clearer safeguards: documented hardware/browser/camera settings, explicit reporting of camera constraints and actual camera settings, fixed screen-brightness settings, pre-session camera/lighting checks, landmark-availability warnings, camera-derived brightness-change records, and specified quality-aware analysis rules. If privacy-sensitive raw video is not stored, these safeguards become more important because many artifacts cannot be retrospectively inspected from images. For studies that aim to make claims about physiological responses, a reference physiological sensor and a multi-participant design are necessary.

6.5 Limitations

This study has strict limitations. First, it was an exploratory N=1 deployment with the author as participant. Second, there was no reference physiological sensor, ECG, or contact PPG. Third, the deployment used one primary laptop/camera configuration and did not test multiple devices, multiple participants, or multiple home environments. Fourth, lighting, screen light, posture, camera placement, task timing, participant state, and task context were not experimentally separated. Fifth, no comparison with existing experiment runtimes, browser rPPG tools, or cloud APIs was conducted. Sixth, the current logs do not store continuous confidence values from the face-processing pipeline for all frames. Therefore, failed landmark-detection frames and overall tracking-success rates cannot be recovered. Future versions should log continuous confidence values for

all frames and apply specified operational thresholds during analysis. Seventh, the study does not claim clinical or medical accuracy, general deployability, general feasibility, robustness, or validated cognitive, motivational, psychological, or physiological effects. The present study presents an implementation example of browser-native, task-aligned, local-first feature logging under limited laboratory and home conditions.

7. Conclusion

We presented Camerala, a browser-native, local-first implementation example that integrates psychophysical task execution, face-derived feature extraction, task-event/frame-record timestamp alignment, operational landmark-availability checking, and local CSV/ZIP persistence without raw facial video retention. In a limited 10-session N=1 author-participant deployment across one laboratory setting and one home setting, the system completed 600 trials under the tested configuration without raw video transmission by Camerala. After the specified reaction-time criterion, 541/600 trials remained for descriptive analysis. The deployment produced synchronized behavioral logs and face-derived and sensor-derived records, and it revealed descriptive deployment-context differences between the tested environments. These observations should not be interpreted as evidence of general deployability, robustness, or physiological/cognitive effects. Instead, Camerala illustrates a task-aligned browser-native implementation and highlights design safeguards needed for future local-first physiological feature logging in home-based psychophysical experiments.

Data availability

The dataset containing facial landmarks, sensor-derived features, and behavioral responses associated with this study cannot be made publicly available due to privacy restrictions and potentially identifiable face-derived data from a domestic deployment context.

CRediT authorship contribution statement

Kotaro Furukawa: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing - original draft.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence this work.

Funding

This work was supported by the College of Transdisciplinary Sciences for Innovation, Kanazawa University.

Reference

[1] Wim Verkruysse, Lars O. Svaasand, and J. Stuart Nelson. Remote plethysmographic imaging using ambient light. *Optics Express*, 16(26):21434–21445, 2008.

[2] Ming-Zher Poh, Daniel J. McDuff, and Rosalind W. Picard. Non-contact, automated cardiac pulse measurements using video imaging and blind source separation. *Optics Express*, 18(10):10762–10774, 2010.

[3] Weixuan Chen and Daniel McDuff. Deepphys: Video-based physiological measurement using convolutional attention networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 349–365, 2018.

[4] A. H. Ayesha, D. Qiao, and F. Zulkernine. A web application for experimenting and validating remote measurement of vital signs. In *Information Integration and Web Intelligence (iiWAS 2022)*, Lecture Notes in Computer Science, vol. 13635, pages 237–251, 2022. doi:10.1007/978-3-031-21047-1_21.

[5] LabVanced. Remote heart rate detection / rPPG. `https://www.labvanced.com/content/technology/en/remote-heart-rate-detection-rPPG/`, accessed July 5, 2026.

[6] K. Wang. FacePhys-Demo: Browser-based rPPG monitor with local inference. `https://github.com/KegangWangCCNU/FacePhys-Demo`, accessed July 5, 2026.

[7] Z. Hasan, E. Dey, S. R. Ramamurthy, N. Roy, and A. Misra. Demo: RhythmEdge: Enabling contactless heart rate estimation on the edge. In *2022 IEEE International Conference on Smart Computing (SMARTCOMP)*, pages 177–179, 2022.

[8] D. Gupta and A. Etemad. Privacy-preserving remote heart rate estimation from facial videos. In *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 706–712, 2023. doi:10.1109/SMC53992.2023.10394350.

[9] D. Di Lernia, G. Finotti, M. Tsakiris, G. Riva, and M. Naber. Remote photoplethysmography (rPPG) in the wild: Remote heart rate imaging via online webcams. *Behavior Research Methods*, 56:6904–6914, 2024. doi:10.3758/s13428-024-02398-0.

[10] K. M. van der Kooij and M. Naber. An open-source remote heart rate imaging method with practical apparatus and algorithms. *Behavior Research Methods*, 51:2106–2119, 2019. doi:10.3758/s13428-019-01256-8.

[11] X. Liu, B. Hill, Z. Jiang, S. Patel, and D. McDuff. EfficientPhys: Enabling simple, fast and accurate camera-based cardiac measurement. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 5008–5017, 2023.

[12] X. Liu, Y. Wang, S. Xie, X. Zhang, Z. Ma, D. McDuff, and S. Patel. MobilePhys: Personalized mobile camera-based contactless physiological sensing. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 6(1), 2022. doi:10.1145/3517225.

[13] X. Liu, G. Narayanswamy, A. Paruchuri, X. Zhang, J. Tang, Y. Zhang, R. Sengupta, S. Patel, Y. Wang, and D. McDuff. rPPG-Toolbox: Deep remote PPG toolbox. *Advances in Neural Information Processing Systems*, 36, Datasets and Benchmarks Track, 2023.

[14] S. Bhutani, M. Elgendi, and C. Menon. Preserving privacy and video quality through remote physiological signal removal. *Communications Engineering*, 4:66, 2025. doi:10.1038/s44172-025-00363-z.

[15] J. R. de Leeuw, R. A. Gilbert, and B. Luchterhandt. jsPsych: Enabling an open-source collaborative ecosystem of behavioral experiments. *Journal of Open Source Software*, 8(85):5351, 2023. doi:10.21105/joss.05351.

[16] J. W. Peirce, J. R. Gray, S. Simpson, M. MacAskill, R. Hoechenberger, H. Sogo, E. Kastman, and J. K. Lindelov. PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, 51:195–203, 2019. doi:10.3758/s13428-018-01193-y.

[17] C. Lugaresi, J. Tang, H. Nash, et al. MediaPipe: A framework for building perception pipelines. arXiv:1906.08172, 2019.

Appendix

Supporting Example: Task Substitution

A core engineering feature of the framework is the separation between task logic and the feature-extraction loop. Listing 1 illustrates a simplified configuration for replacing the orientation task with a spatial-memory task while preserving the same logging pathway. This example is intended to illustrate module separation rather than to provide additional validation data.

Listing 1: Task substitution example

```
const task = {
  stimulus: renderSpatialGrid(memory_load_target),
  allowed_keys: ["Space"],
  on_frame_sync: (timestamp_ms) => {
    camerala_logger.logEvent("stimulus_onset", timestamp_ms);
  }
};
```

Listing 2 shows a schema-style example of a local tabular log in which task events and frame-derived features can coexist through timestamps.

Listing 2: Example local log schema

```
time_ms,event,load,rppg_roi,head_motion
4015.2,grid_on,4,110.4,0.02
4016.8,frame_sync,4,109.8,0.02
4032.5,frame_sync,4,111.1,0.01
4211.4,response,4,112.0,0.01
```